

DECODE: Dynamic Exploration for Constraint-Guided Vulnerability Discovery in Deep Learning Operators

HAOTONG LIU, Nankai University, China

ZHI WANG*, Nankai University, China

ZHUOHANG LIU, Nankai University, China

WANPENG LI, University of Liverpool, United Kingdom

The security and robustness of deep learning (DL) frameworks are vital, as vulnerabilities in low-level operator implementations can lead to serious reliability and security risks. While testing has proven effective in uncovering such flaws, existing techniques struggle to accurately capture the complex input constraints required by DL operators, resulting in low test coverage and missed bugs.

To address this, we propose DECODE, a fully automated framework that performs efficient and precise constraint extraction through dynamic analysis. DECODE models operator-specific input requirements by observing valid execution traces and exploring constraint relationships. It then uses these refined constraints to generate high-quality test inputs capable of exposing memory error vulnerabilities. We evaluated DECODE on two widely used DL frameworks - TensorFlow and PyTorch - where it uncovered 96 bugs (54 in TensorFlow and 42 in PyTorch), 82 of which have been confirmed by developers. Compared to state-of-the-art tools, DECODE detected 41, 75, and 87 more bugs than IvySyn, DocTer, and DeepREL, respectively, demonstrating its superior effectiveness in uncovering previously undetected vulnerabilities.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Deep Learning Library Testing, Constraint Extraction, Memory Error, Dynamic Exploration

ACM Reference Format:

Haotong Liu, Zhi Wang, Zhuohang Liu, and Wanpeng Li. 2026. DECODE: Dynamic Exploration for Constraint-Guided Vulnerability Discovery in Deep Learning Operators. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE167 (July 2026), 23 pages. <https://doi.org/10.1145/3808174>

1 Introduction

Deep Learning [21] (DL) has become an integral part of modern software systems, fueling advancements across various industries. Popular DL libraries such as TensorFlow [3] and PyTorch [29] have become the backbone of numerous applications, from image recognition [17] and natural language processing [6, 24, 42] to reinforcement learning [36, 44]. However, errors in these DL libraries can severely compromise system integrity, leading to incorrect model behavior, service disruptions, or security vulnerabilities, such as denial-of-service attacks [1, 30], unauthorized code execution [2], or crashes caused by unhandled exceptions.

*Zhi Wang is the corresponding author.

Authors' Contact Information: [Haotong Liu](#), College of Cryptology and Cyber Science, Nankai University, Tianjin, China, lht@mail.nankai.edu.cn; [Zhi Wang](#), College of Cryptology and Cyber Science, Nankai University, Tianjin, China, zwang@nankai.edu.cn; [Zhuohang Liu](#), College of Cryptology and Cyber Science, Nankai University, Tianjin, China, zhuohangliu@mail.nankai.edu.cn; [Wanpeng Li](#), School of Computer Science and Informatics, University of Liverpool, Liverpool, United Kingdom, wanpeng.li@liverpool.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE167

<https://doi.org/10.1145/3808174>

By automating the generation and mutation of input data, traditional fuzzing [12, 27, 32, 34, 51] has proven effective in uncovering bugs, performance bottlenecks, and implementation errors in software. However, recent studies [5, 11, 15, 46–48] reveal that traditional fuzzing is not well-suited for DL libraries due to their unique computational characteristics. And existing DL testing techniques still face significant challenges:

C1: Extracting Accurate Operator Input Constraints. Deep learning operators enforce complex input constraints through low-level assertion checks in their implementations [50]. Current approaches rely on documentation[48] or static analysis[35] to infer these constraints. However, such inferred constraints often fail to precisely capture the assertion checks enforced in operator implementations. Consequently, test generation guided by such imprecise constraints produces inputs that either violate implementation-level checks or fail to exercise critical boundary conditions, fundamentally limiting effective exploration of operator behaviors and bug discovery.

C2: Modeling Cross-Tensor Dependencies. Operator input validation frequently imposes precise cross-tensor relationships that constrain multiple inputs simultaneously. For example, matrix multiplication requires dimensional compatibility: tensor A’s last dimension must equal tensor B’s first dimension, such that $A[m, k] \times B[k, n]$ is only valid when the inner dimension k matches. Existing testing approaches typically treat operator constraints monolithically without explicitly modeling such cross-tensor dependencies. Consequently, generated inputs often violate these cross-tensor constraints and are prematurely rejected by assertion checks. This results in low valid-input rates and severely limits exploration of cross-tensor boundary behaviors where complex bugs frequently emerge.

C3: Detecting Assertion Logic Defects. Assertion checks are typically assumed to correctly safeguard backend kernels against invalid inputs. However, assertion logic itself can be defective: assertions may be missing for certain input conditions, positioned incorrectly in the control flow, or implement flawed validation predicates that cannot handle specific input combinations. When such defects exist, unsafe inputs trigger crashes directly within the assertion logic during validation, as the defective assertions fail to correctly process adversarial inputs. Since existing testing approaches treat assertions as ground truth when extracting and enforcing input constraints, they are fundamentally unable to detect bugs originating from faulty validation logic itself, leaving a critical class of vulnerabilities unexplored.

Our Approach. To address the above challenges, we propose DECODE (**D**ynamic **E**xploration for **C**onstraint-Guided **V**ulnerability **D**iscovery in **D**Eep Learning Operators), a fully automated testing approach to detect vulnerabilities in DL operators. DECODE dynamically explores and models operator input constraints for DL operators and leverages them to generate high-quality test cases.

To address **C1**, DECODE models operator constraints as evolving hypotheses rather than static specifications. Starting from initial constraints extracted from valid executions, DECODE iteratively refines them using runtime feedback from assertion violations, relaxing overly restrictive constraints and strengthening under-specified ones. This dynamic refinement aligns constraint models with actual validation logic, enabling generation of semantically valid inputs that exercise meaningful boundary behaviors.

To address **C2**, DECODE explicitly distinguishes cross-tensor constraints from tensor-specific constraints representing cross-tensor relationships (e.g., dimensional compatibility) as distinct constraint categories. By identifying and selectively breaking unintended constraint inference chains that couple tensor dimensions, DECODE prevents premature input rejection, significantly improving valid-input rates and enabling systematic exploration of multi-tensor boundary cases.

To address **C3**, DECODE treats assertion logic as a testable component rather than a trusted specification. DECODE systematically generates inputs that stress assertion validation predicates,

including boundary cases and adversarial combinations that may expose flawed logic. By analyzing crash locations, DECODE distinguishes failures in assertion code from those in backend kernels. Crashes during assertion execution reveal defective validation predicates, missing edge-case checks, or incorrect validation ordering, a critical vulnerability class that existing approaches fundamentally cannot detect since they assume assertion correctness when modeling constraints.

We evaluated the performance of DECODE on two mainstream DL libraries, TensorFlow and PyTorch. DECODE demonstrates its efficacy in constraint extraction, achieving precision, recall, and F1 scores of 80.6%/81.0%/80.8% for TensorFlow and 74.9%/78.3%/76.6% for PyTorch, highlighting its capability to model input constraints and generate high-quality test cases efficiently. Thanks to the dynamic exploration, DECODE's extracted constraints significantly enhanced input passing rates, reaching 40.1% for TensorFlow and 36.2% for PyTorch. DECODE successfully identified 96 bugs (54 in TensorFlow and 42 in PyTorch), outperforming IvySyn[5], DocTer[48], and DeepREL[11]. We define bugs as crash-inducing inputs that trigger memory errors in operator implementations. Of the 96 bugs, 82 were confirmed by developers as requiring fixes, representing 47 unique defects. The remaining 14 were declined not due to false positives or irreproducibility, but because maintainers deemed the triggering conditions (e.g., extreme tensor dimensions, boundary parameter values) outside nominal usage scenarios. These nonetheless constitute genuine bugs as they expose unhandled edge conditions that can trigger crashes or memory corruption in real-world deployments.

This paper makes the following contributions:

- We propose a novel dynamic exploration-based method for modeling input constraints of DL operators. By observing valid inputs during execution, our approach automatically extracts operator constraints with minimal testing overhead. Unlike general-purpose solvers such as Z3, which struggle with tensor-specific, variable-length constraints, our method is tailored to the structure of DL operators, yielding more efficient and precise constraint extraction.
- We present DECODE, a dynamic exploration-driven testing framework designed to model input constraints for DL operators and optimize test case generation. By identifying the essential constraints governing operator inputs, DECODE enables the creation of diverse, high-quality test cases that improve bug detection capabilities and uncover subtle vulnerabilities.
- We evaluated DECODE on two DL libraries, TensorFlow and PyTorch. Leveraging the extracted constraints, DECODE generates high-quality test cases that reveal hidden errors in operator implementations, identifying a total of 96 bugs, 82 of which were confirmed by the developers. DECODE significantly outperformed state-of-the-art bug detectors Ivysyn, DocTer, and DeepREL, uncovering 47 unique bugs.

2 Background and Motivation

2.1 DL Library Testing

Errors [22, 47] in DL libraries can propagate and cause inaccuracies, inefficiencies, or failures in safety-critical applications. Thus, rigorous testing is crucial to ensuring DL libraries' reliability. Existing DL library testing techniques can be divided into two categories: model-level testing and API-level testing.

Model-level testing [15, 16, 23, 26, 31, 43, 45] evaluates DL libraries by generating and executing models to assess various aspects of the library's functionality. By focusing on the handling of entire models, model-level testing aims to ensure that libraries can reliably support complex workflows, detect errors, and optimize performance. Approaches like Muffin [15] abstract models as Directed Acyclic Graphs with chain and cell-based structures to enhance layer API coverage, while COMET [23] uses mutation operators and a coverage-based search algorithm to diversify input data, parameters, and layer sequences.

API-level testing [5, 10, 11, 13, 35, 46, 48] focuses on validating individual functions or operators in DL libraries and can be categorized into high-level API testing and operator testing. High-level testing evaluates user-facing APIs, such as TensorFlow’s Keras and PyTorch’s `torch.nn`. Several approaches have been proposed. FreeFuzz [46] tests API functions by leveraging previously observed valid invocations from available resources. DeepREL [10] generates valid inputs for functions lacking seed inputs by exploiting the similarity between API functions, transferring inputs from test cases of related functions to bridge the gaps. Operator-level testing focuses on discovering bugs and performance issues in kernels. Approaches like IvySyn [5] perform type-aware mutation-based testing on kernel code. In addition ACETest [35] employs static analysis to guide input generation by extracting path constraints.

Recent advancements in large language models (LLMs), such as ChatGPT [4], have opened new avenues for DL library testing. TitanFuzz [8] uses LLMs to automate generation and mutation fuzzing, improving coverage and bug detection in DL libraries. FuzzGPT [9] further improves fuzzing with fine-tuning and in-context learning to automate program generation. DeepConstr [13] applies LLM for input constraint refinement with set-guided and genetic methods, boosting testing completeness and coverage.

2.2 Motivation

Deep learning operators typically enforce prerequisites (e.g., assertion checks) to ensure correct execution. These requirements impose constraints on input tensors—such as shapes, dimensions, data types, and inter-tensor relationships—reflecting the assumptions underlying operator computation. While static analysis can extract such constraints from source code, it has inherent limitations. In particular, the extracted constraints may encode flawed logic in the implementation. Consequently, test inputs generated from these constraints may miss critical edge cases, including those that expose bugs within the assertion logic itself.

```

1 class DilationBackpropFilterOp : public OpKernel {
2 public:
3 void Compute(OpKernelContext* context) override {
4     const Tensor& input = context->input(0);
5     const Tensor& filter = context->input(1);
6     const Tensor& out_backprop = context->input(2);
7     ...
8     OP_REQUIRES(context,
9         batch == out_backprop.dim_size(0) &&
10         out_rows == out_backprop.dim_size(1) &&
11         out_cols == out_backprop.dim_size(2) &&
12         depth == out_backprop.dim_size(3),
13         errors::InvalidArgument("out_backprop has incompatible
14                                 size.));
15 }
16 };

```

Listing 1. Crash Location in Assertion Check. Yellow Highlighting Indicates Crash Location.

For example, in Listing 1, static analysis may infer a constraint such as `depth == out_backprop.dim_size(3)`, leading to the assumption that `out_backprop` must be at least four-dimensional. Consequently, test inputs generated under this assumption will never include `out_backprop` tensor with fewer than four dimensions. However, `DilationBackpropFilter` operator implicitly assumes `out_backprop` has four dimensions but does not explicitly check for this. If a three-dimensional `out_backprop` is passed, the assertion tries to access a non-existent fourth

```

1 Operator: "FusedPadConv2D"
2 Inputs:
3   - Name: "input"
4     Type Attribute: "T"
5   - Name: "padding"
6     Type: DT_INT32
7   - Name: "filter"
8     Type Attribute: "T"
9 Attributes:
10  - Name: "T"
11    Type: "type"
12    Allowed Values: { DT_HALF, DT_FLOAT,
13                     DT_DOUBLE}

```

Listing 2. FusedPadConv2D parameters description. Note: *input* and *filter* must have the same data type *T*.

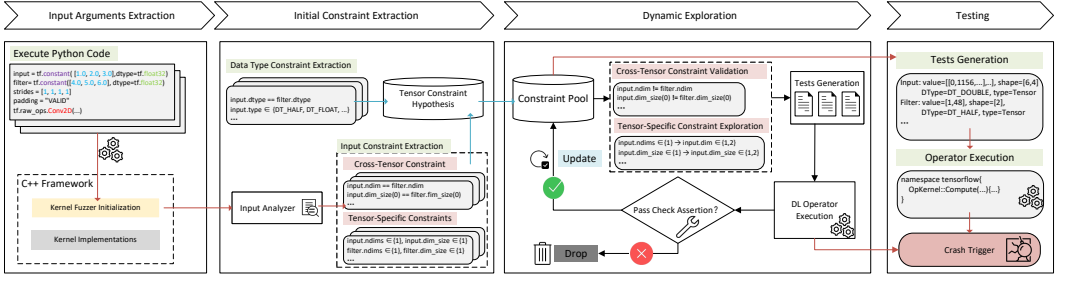


Fig. 1. Overview of DECODE

dimension, leading to a crash in the C++ backend (highlighted in yellow in the listing), exposing a hidden flaw in the assertion logic.

This example reveals a critical weakness in the validation process: assertion checks often enforce narrow, condition-specific constraints without verifying all underlying input assumptions. As a result, these partial checks leave gaps that allow subtle yet serious bugs to go undetected. Relying solely on static analysis to extract constraints further exacerbates this issue. Static analysis may infer constraints from flawed or incomplete logic—e.g., assuming a tensor must be four-dimensional simply because its third dimension is accessed—without verifying whether such assumptions are enforced. This introduces a form of confirmation bias in test generation, where only inputs satisfying these inferred constraints are produced, limiting the exploration of critical edge cases. Static constraint extraction therefore provides only partial coverage, with the following limitations:

- **Narrow Validation Scope:** Assertion checks validate explicit conditions but often miss hidden dependencies or edge cases. For example, a check may compare tensor dimensions without ensuring their existence (Listing 1), leaving gaps where violated assumptions go undetected.
- **Faulty Assertions:** Assertions themselves may be incorrect, dereferencing invalid dimensions or relying on wrong assumptions, leading to crashes. Static analysis cannot reliably distinguish such faulty assertions, limiting its effectiveness for constraint construction.

These limitations highlight the need for a dynamic approach that not only extracts but also refines constraints based on actual operator behavior, enabling the discovery of deeper bugs.

3 Approach

Figure 1 illustrates the DECODE workflow, which consists of four stages. First, DECODE intercepts and analyzes input data from Python test code before execution reaches the operator’s C++ implementation, extracting input argument details. Next, it derives data type constraints from operator descriptions provided by deep learning frameworks. Based on the extracted argument details and data type constraints, it formulates tensor constraint hypotheses. In the third stage, DECODE refines these hypotheses by validating both Cross-Tensor and Tensor-Specific constraints, ensuring comprehensive constraint coverage. Finally, the refined constraints guide the generation of diverse test inputs that satisfy logical assertions while effectively probing operator implementations for potential bugs.

3.1 Input Arguments Extraction

Our work draws inspiration from IvySyn [5], recognizing that testing at the implementation level of C++ operators in DL libraries offers significant efficiency advantages. By directly targeting the

operator implementations, we bypass the overhead of Python-to-C++ data transmission and the additional complexity of multi-layer function calls.

We initiate the process by executing Python test code[33, 41] that invokes APIs from DL libraries. Since these libraries implement functional operators in C++, we integrate fuzzing-related code into the C++ implementation to intercept execution just before the operator's function is invoked. This allows us to capture the operator's original inputs. For instance, Figure 2(a) shows the Python-level input for the FusedPadConv2D in TensorFlow. When this input is passed to its C++ implementation, we capture its transformed C++ format, as shown in Figure 2(b).

Our methodology assumes that inputs from the Python test code will successfully pass the assertion checks embedded within the operator's C++ implementation. These checks validate inputs against predefined criteria, such as dimensionality, data type, and value ranges. Inputs that meet these conditions inherently conform to the constraints defined by the operator. By capturing these valid inputs, we establish a foundation for further constraint verification.

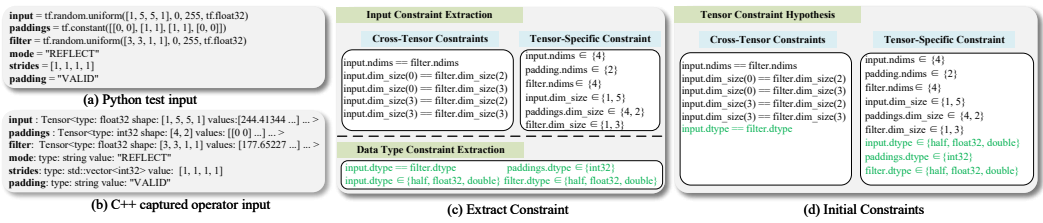


Fig. 2. Initial Constraint Extraction for FusedPadConv2D Operator in Tensorflow. Constraints highlighted in green represent Data Type Constraints directly extracted from the DL framework's operator descriptions (Section 3.2.1), which do not require refinement during the dynamic exploration phase (Section 3.3).

3.2 Initial Constraint Extraction

Prior approaches [35, 46, 48] focused on constraint analysis of tensor properties (see Table 1), such as dimension and dim_size, which are primarily used to impose restrictions on input parameters. Building on this, we integrate tensor property descriptions with operator assertion checks to construct a comprehensive operator constraint model.

Table 1. Properties of Tensors in Deep Learning Libraries

Property	Description
ndims	Describes the number of dimensions
dtype	Describes the data type of the elements
dim_size	Describes the size of the tensor in each dimension
num_elements	Describes the number of elements

Constraint Model Definition. Our approach models the complex constraints of DL operators by analyzing their implementation at the C++ level. From this analysis, we derive the operator's constraints into a structured representation, summarized in Table 2:

- **Cross-Tensor Constraints:** These constraints stem from operator assertions that verify relationships between input tensors, capturing inter-dependencies among tensors.
- **Tensor-Specific Constraints:** These constraints specify the requirements for individual tensor properties, such as number of elements, dimensions, data types, and other attributes specific to each tensor.

DECODE leverages captured valid inputs to construct initial constraints for DL operator inputs based on the Tensor Constraint Model shown in Table 2. First, it extracts tensor data type constraint information from the operator descriptions provided by DL frameworks (3.2.1). Then, it analyzes valid inputs (3.2.2) to map input features to the constraints defined by the Tensor Constraint Model.

Table 2. Tensor Constraint Model. Cross-Tensor Constraints refer to the relationships between multiple tensors, while Tensor-Specific Constraints refer to the requirements or conditions that apply individually to each tensor. Note that the *num_elements* property is not listed separately in Tensor-Specific Constraints as the combination of *ndims* and *dim_size* can represent it.

Constraint Type	Property	Description	Example
Cross-Tensor Constraints	<i>dtype</i>	dtype matching between tensors	<code>tensor1.dtype == tensor2.dtype</code>
	<i>ndims</i>	number of dimensions matching between tensors	<code>tensor1.ndims == tensor2.ndims</code>
	<i>num_elements</i>	number of elements matching between tensors	<code>tensor1.num_elements == tensor2.num_elements</code>
	<i>dim_size</i>	specific dimensions matching between tensors	<code>tensor1.dim_size(0) == tensor2.dim_size(2)</code>
Tensor-Specific Constraints	<i>dtype</i>	dtype range of tensor	<code>tensor1.dtype ∈ {int32, float64}</code>
	<i>ndims</i>	number of dimensions range of tensor	<code>tensor1.ndims ∈ {1, 2, 3}</code>
	<i>dim_size</i>	specific dimension range of tensor	<code>tensor1.dim_size(0) ∈ {1, 10, 100}</code>

3.2.1 Data Type Constraint Extraction. DL libraries typically provide operator description interfaces that specify the parameter details of operator APIs. From these operator descriptions, we extract critical information, including the number of parameters, parameter types, and the data type (*dtype*) constraints associated with tensor-type parameters. In TensorFlow, we extract details of operator parameters using `tensorflow::GetAllRegisteredKernels`, while in PyTorch¹, we leverage `Torchgen.model.NativeFunctionsGroup`.

To illustrate this process, consider the `FusedPadConv2D` operator in TensorFlow. As shown in Listing 2, this operator has three parameters: *input*, *padding*, and *filter*. By analyzing the parameter *dtype* descriptions, we derive the following example constraints:

- Cross-Tensor Constraint: Since *input* and *filter* share the same *type_attr*, their dtypes must match, expressed as `input.dtype == filter.dtype`.
- Tensor-Specific Constraint: The *dtype* of both the *input* and *filter* parameters is restricted to `{DT_HALF, DT_FLOAT, DT_DOUBLE}`.

This example illustrates how data type constraints for both Tensor-Specific and Cross-Tensor Constraints are derived. These constraints are directly extracted from the DL API, ensuring their reliability. As a result, they remain unchanged during the constraint exploration phase (see section 3.3).

3.2.2 Tensor Constraint Hypothesis. After obtaining data type constraints, we also analyze other tensor attributes - such as *ndims*, *dim_size*, as shown in Table 1 - to derive additional Cross-Tensor and Tensor-Specific Constraints enforced in operator implementations.

We assume that inputs from Python test cases, when accepted by an operator, successfully pass all operator-level assertion checks without errors. This assumption is reasonable, as these test cases are primarily designed to verify operator functionality rather than trigger failures. Based on this foundation, we generate tensor constraint hypotheses which serve as the basis for our dynamic constraint exploration (see Section 3.3).

¹Unlike TensorFlow, PyTorch lacks explicit input tensor type declarations. Instead, type validations are typically enforced through runtime assertions. DECODE infers data type constraints by capturing and analyzing runtime input values. To enhance generality and improve test diversity, the observed data types are abstracted into broader type families - for example, an input of type `int32` is generalized to include the entire integer family: `int16`, `int32`, and `int64`.

To achieve this, we extract key tensor attributes such as `ndims`, `dim_size`, and `num_element` (as listed in Table 2) and map them into Cross-Tensor Constraints and Tensor-Specific Constraints based on the input arguments extracted in section 3.1. These mappings are then combined with the previously derived Data Type Constraints (Section 3.2.1) to construct a comprehensive Tensor Constraint Hypothesis.

As illustrated in Figure 2(c), we extract constraints for tensor properties such as `ndims`, `dim_size`, and `num_element` during the Input Constraint Extraction process. By combining these with the data type constraints extracted in Section 3.2.1, we formulate the Tensor Constraint Hypothesis. This hypothesis serves as an initial approximation of the operator's input constraints and requires further exploration.

3.3 Dynamic Exploration

In this section, we validate and refine the tensor constraint hypotheses extracted previously to ensure their accuracy and completeness. Since official test cases primarily verify operator correctness rather than covering all inputs and edge cases, the derived constraints are treated as hypotheses rather than definitive rules. For example, a test case may suggest a relation such as $x.ndims = y.ndims$, which does not necessarily correspond to an enforced Cross-Tensor Constraint. Similarly, a single test case may impose overly restrictive Tensor-Specific Constraints, limiting `ndims` and `dim_size` to narrower ranges than necessary. Misinterpreting these patterns as strict constraints reduces test diversity and coverage. Relying solely on such hypotheses may yield a narrow test set, preventing the generation of inputs that expose memory errors beyond assertion checks.

To address this limitation and refine the extracted constraints for generating high-quality test cases — ones that both satisfy assertion checks and effectively trigger memory errors — we introduce a two-step dynamic exploration phase. This approach leverages execution feedback to iteratively refine constraint-guided inputs and ensure constraint validity. First, DECODE validates Cross-Tensor Constraints (Section 3.3.1), as constraints extracted from test cases may not reflect actual operator restrictions. These hypotheses are verified against assertion checks to ensure their reliability for test generation. Next, DECODE explores the value ranges of Tensor-Specific Constraints (Section 3.3.2) to further refine constraint boundaries, improving coverage and the likelihood of uncovering deep-seated vulnerabilities.

3.3.1 Cross-Tensor Constraint Validation. For Cross-Tensor Constraints, we need to validate whether the extracted input constraints are actually enforced by the operator's input assertion checks.

Let C_{check} represent the constraint under validation, $\neg C_{check}$ its negation, and T the set of test inputs generated based on $\neg C_{check}$. The validation proceeds as follows:

- (1) The test input set T is executed on the operator O to evaluate C_{check} .
- (2) If T passes without triggering assertion failures, it indicates that C_{check} is not enforced by the operator.
- (3) Conversely, if T fails due to assertion violations, C_{check} is confirmed as a necessary and effective constraint.

However, our research finds that directly negating C_{check} to generate T is not always feasible due to implicit inference, where C_{check} can be derived from other constraints, preventing the generation of valid test inputs T . We categorize implicit inference into two types: intra-property constraint inference and cross-property constraint inference.

- **Intra-Property Constraint Inference.** This type of inference arises when relationships within the same constraint category, such as Cross-Tensor Constraints for specific attributes (e.g., `ndim`, `dim_size`), lead to derived equivalences. As shown in Figure 3(a), tensors A ,

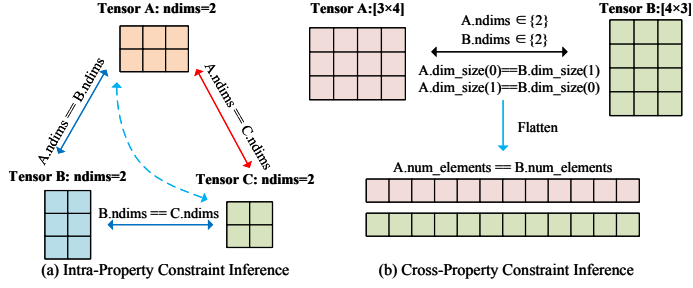


Fig. 3. Examples of Implicit inference

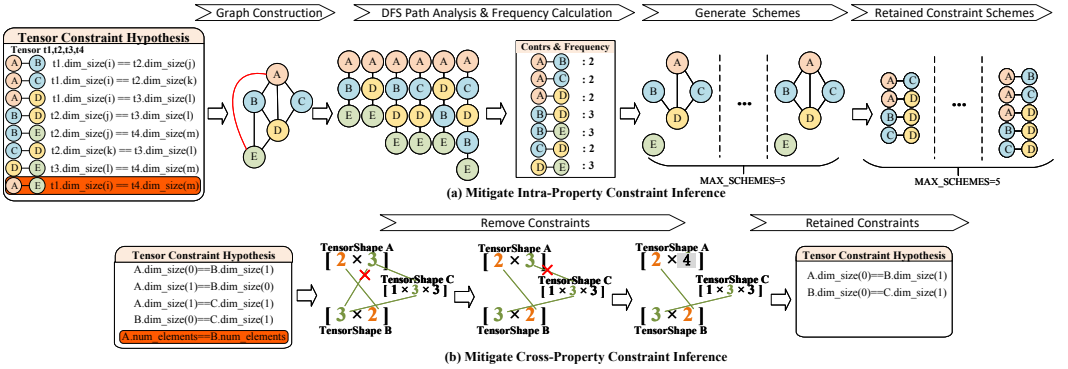


Fig. 4. Examples of Mitigating Conflicts Arising from Implicit Constraint Inference. Figure (a) shows how DECODE mitigates intra-property constraint inference. The process produces up to $MAX_SCHEMES$ constraint sets that eliminate indirect inference within intra-property constraints. Figure (b) illustrates the resolution of cross-property constraint inference, where constraints marked with “ $\times$ ” are randomly removed to break unintended inference paths. The constraints enclosed in the red box represent the specific constraint currently being evaluated.

B , and C share a constraint where their dimensions must be equal. Given the constraints $A.ndims == B.ndims$ and $B.ndims == C.ndims$, we can infer $A.ndims == C.ndims$. Checking such inferred constraint C_{check} (e.g., $A.ndims == C.ndims$) makes it impossible to generate test cases for $\neg C_{check}$.

- **Cross-Property Constraint Inference.** This occurs when Cross-Tensor Constraints on the `num_elements` property can be deduced from existing constraints on the `ndims` and `dim_size` properties. As shown in Figure 3(b), if two tensors have identical values for `ndims` and `dim_size` attributes across all dimensions, it is logically inferred that their `num_elements` values must also be equal.

To mitigate conflicts caused by implicit constraint inference during Cross-Tensor Constraint validation, we represent tensor constraints as an undirected graph, where nodes correspond to tensor properties and edges denote constraints. For example, as illustrated in Figure 4(a), node A represents the attribute `dim_size(i)` of tensor $t1$, while node B represents `dim_size(j)` of tensor $t2$. An edge between them indicates the constraint $t1.dim_size(i) == t2.dim_size(j)$.

We use the example in Figure 4(a) to demonstrate how to eliminate conflicts caused by Intra-Property Constraint Inference using Algorithm 1. We first construct an undirected graph from Cross-Tensor Constraints derived from the constraint hypotheses (Line 1). The target constraint,

Algorithm 1. Mitigate Intra-Property Constraint Inference

Require: Checked Constraint *ConstraintCheck*
Require: Cross-Tensor Constraints *ConstraintSet*
Ensure: Retained Schemes *retained_schemes*

```

1:  $G \leftarrow \text{BUILDUNDIRECTEDGRAPH}(\text{ConstraintSet})$ 
2:  $\text{begin\_node}, \text{end\_node} \leftarrow \text{GETNODE}(G, \text{ConstraintCheck})$ 
3:  $\text{paths} \leftarrow \text{FINDALLPATHS}(G, \text{begin\_node}, \text{end\_node})$ 
4:  $\text{edges}, \text{frequency} \leftarrow \text{CALCULATE}(\text{paths})$ 
5:  $\text{retained\_schemes} \leftarrow \emptyset$ 
6:  $\text{tmp\_graph} \leftarrow \text{COPYGRAPH}(G)$ 
7: for  $i = 1$  to  $\text{MAX\_SCHEMES}$  do
8:    $G \leftarrow \text{COPYGRAPH}(\text{tmp\_graph})$ 
9:    $\text{removed\_scheme} \leftarrow \emptyset$ 
10:  while  $\text{IsCONNECTED}(G, \text{begin\_node}, \text{end\_node})$  do
11:     $\text{edge} \leftarrow \text{SELECT}(\text{edges}, \text{frequency})$ 
12:     $\text{REMOVE}(G, \text{edge})$ 
13:     $\text{constr} \leftarrow \text{EDGETOCONSTRAINT}(\text{edge})$ 
14:     $\text{removed\_scheme} \leftarrow \text{removed\_scheme} \cup \{\text{constr}\}$ 
15:  end while
16:   $\text{scheme} \leftarrow \text{SETDIFFERENCE}(\text{ConstraintSet}, \text{removed\_scheme})$ 
17:   $\text{retained\_schemes} \leftarrow \text{retained\_schemes} \cup \{\text{scheme}\}$ 
18: end for
19: return  $\text{retained\_schemes}$ 

```

Algorithm 2. Cross-Tensor Constraint Validation

Require: Cross-Tensor Constraints *ConstraintSet*
Ensure: Validated Constraints *validConstraints*

```

1:  $\text{validConstraints} \leftarrow \emptyset$ 
2: for  $\text{constr} \in \text{ConstraintSet}$  do
3:    $\text{ToBeVerified} \leftarrow \{\neg \text{constr}\}$ 
4:   if  $\text{EXISTINFERENC}(\text{ToBeVerified})$  then
5:      $\text{schemes} \leftarrow \text{SOLVE}(\text{ToBeVerified})$ 
6:      $\text{ToBeVerified} \leftarrow \text{ToBeVerified} \cup \{\text{RANDOM}(\text{schemes})\}$ 
7:   end if
8:    $\text{tests} \leftarrow \text{GENERATOR}(\text{ToBeVerified})$ 
9:   if  $\text{NOTCHECKPASS}(\text{tests})$  then
10:     $\text{validConstraints} \leftarrow \text{validConstraints} \cup \{\text{constr}\}$ 
11:   end if
12: end for
13: return  $\text{validConstraints}$ 

```

$t1.\text{dim_size}(i) == t4.\text{dim_size}(m)$, is highlighted in Figure 4(a). We then apply DFS [39] to enumerate reachable paths and compute edge frequencies (Line 3). To violate this constraint, DECODE iteratively removes constraints to disrupt inference, prioritizing high-frequency edges that are critical for connectivity. In each of the *MAX_SCHEMES* iterations (Lines 7–16), edges are removed until the graph becomes disconnected between the *begin_node* ($t1.\text{dim_size}(i)$) and *end_node* ($t4.\text{dim_size}(m)$) (Lines 10–15). Each removed edge corresponds to a constraint. Once disconnected, the current scheme is computed as the set difference between all constraints and the removed ones (Line 15), and added to *retained_schemes* (Line 16). We set *MAX_SCHEMES* = 5 to limit iterations.

We use the example in Figure 4(b) to illustrate how to resolve conflicts from cross-property constraint inference. The constraint under validation is $A.\text{num_elements} == B.\text{num_elements}$. The graph includes $A.\text{dim_size}(0) == B.\text{dim_size}(1)$ and $A.\text{dim_size}(1) == B.\text{dim_size}(0)$, forming an indirect inference path: matching dimensions between *A* and *B* allow deducing equal numbers of elements. In practice, such inference chains may involve many tensors and constraints, increasing complexity. To eliminate this implicit inference, we iteratively remove contributing constraints (e.g., $A.\text{dim_size}(1) == B.\text{dim_size}(0)$) using Algorithm 1 until the graph is disconnected between the *begin_node* ($A.\text{dim_size}(1)$) and *end_node* ($B.\text{dim_size}(0)$). If alternative paths exist (e.g., via a tensor *C*), additional constraints are removed accordingly. Once disconnected, the remaining constraints (e.g., $B.\text{dim_size}(0) == C.\text{dim_size}(1)$ and $A.\text{dim_size}(0) == B.\text{dim_size}(1)$) can no longer infer $A.\text{num_elements} == B.\text{num_elements}$.

Algorithm 2 outlines the overall process for Cross-Tensor Constraint Validation. For each constraint, we first check whether the constraint is influenced by implicit inference (Line 4). If implicit inference is detected, we use the methods introduced above to identify and retain the constraints that remain after eliminating the inference (Lines 5–6). These constraints are then used to generate test inputs that satisfy $\neg C_{\text{check}}$ (Line 8). Next, the operator is executed with these generated inputs. If the inputs trigger assertion failures, the constraint is confirmed as necessary and retained for test generation (Lines 9–10).

3.3.2 Tensor-Specific Constraint Exploration. After validating Cross-Tensor Constraints, we refine Tensor-Specific Constraints by exploring the value ranges of key tensor properties, such as *ndims*

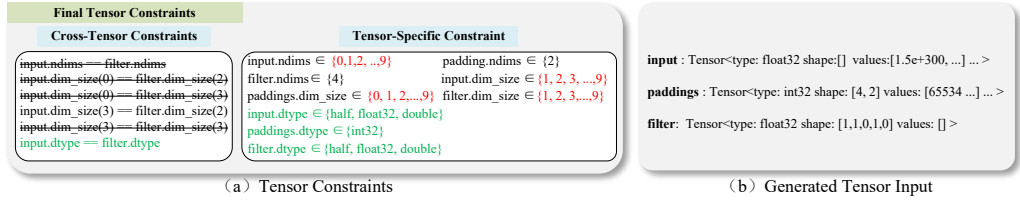


Fig. 5. Generating test inputs for FusedPadConv2D Operator. (a) illustrates the refined constraint set after Dynamic Exploration. Constraints marked with strikethrough were invalidated during exploration, while those highlighted in red denote Tensor-Specific Constraints whose value ranges were extended. (b) provides a representative example of tensor inputs generated from the refined constraints in (a).

and `dim_size`. This process ensures that the extracted constraints accurately reflect the operator's actual input bounds, thereby improving the diversity and effectiveness of the generated test data.

During this exploration, it is essential to preserve Cross-Tensor Constraints. For example, if a constraint enforces $x.ndims == y.ndims$, we must avoid generating inputs where $x.ndims$ falls outside the valid range of $y.ndims$. To maintain these relationships, we group tensors based on their Cross-Tensor Constraints using the union-find algorithm [40]. This approach ensures that tensors like x and y , which share a constraint $x.ndims == y.ndims$, are explored collectively, preventing violations while refining their value ranges.

The refinement process proceeds group by group, analyzing the `ndims` and `dim_size` properties within each tensor group. For each potential property update, its validity is checked against the operator assertion. If the assertion passes, the value is added to the feasible set.

After validating Cross-Tensor Constraints and refining the value ranges of Tensor-Specific Constraints, we proceed to the final testing phase.

3.4 Testing

Building on insights from IvySyn [5] and Orion [37], which emphasize that bugs often arise from edge-case inputs. We systematically explore such scenarios. To achieve this, we introduce extreme values into input parameters, including null values, maximum and minimum permissible values, and other boundary conditions. The refined constraint set from the previous stage serves as the foundation for generating test cases. This ensures that the generated inputs adhere to the operator's constraints while deliberately probing edge scenarios. As illustrated in Figure 5, we use the refined constraints in Figure 5(a) to guide the generation of Tensor attributes such as shape and dtype. The generated test cases are then executed on the operator to evaluate its behavior. This process aims to identify potential bugs or failures triggered by edge-case inputs, thereby enabling a comprehensive assessment of the operator's robustness and stability under varied conditions.

4 Implementation

DECODE is implemented with approximately 4,000 lines of C/C++ and 200 lines of Python, building on the IvySyn approach by conducting tests directly on C/C++ operator implementations. The full source code is released at <https://github.com/DECODE-paper/DECODE>, and identified issues have been reported to the PyTorch and TensorFlow repositories.

Fuzzing logic is embedded into operator-level implementations. In TensorFlow, we use libclang [20] 11.1.0 to wrap low-level kernels (e.g., `OpKernel` and `AsyncOpKernel`), while in PyTorch, we leverage the existing code generation mechanism.

Before input argument extraction, we analyze Python test scripts from both frameworks to ensure correct execution with valid inputs. Operator parameter specifications are extracted using platform-specific tools (e.g., `tensorflow::GetAllRegisteredKernels` and `Torchgen.model.NativeFunctionsGroup`) and mapped to our Tensor Constraint Model to construct Tensor Constraint Hypotheses (Section 3.2.1), including data type, tensor-specific, and cross-tensor constraints.

During dynamic exploration, DECODE generates 10 test inputs per iteration to balance efficiency and diversity. All experiments are conducted on CPUs, with each operator tested for 10 minutes to evaluate robustness and detect bugs.

5 Evaluation

We evaluate DECODE by answering the following research questions:

RQ1 How effective is DECODE in extracting constraints in Dynamic Exploration phase?

RQ2 Do the extracted constraints enable DECODE to generate high-quality test cases effectively?

RQ3 Can DECODE effectively identify previously unknown bugs in real-world DL operators?

5.1 Experimental Setup

5.1.1 Target DL Libraries. We conducted our evaluation of DECODE on both PyTorch (v2.5.0) [29] and TensorFlow (v2.16.1) [3]. To ensure a robust and practical analysis, we applied a set of exclusion criteria to filter operators based on the following three rules: ① operators that do not accept tensor-type inputs, ② operators that require inputs such as resource handles, variant types, or other non-tensor structures, and ③ operators that are not running on CPU. This approach ensures a focus on operators that are broadly applicable and manageable within the libraries. After applying the filtering criteria, we conducted testing on 583 operators from TensorFlow and 710 operators from PyTorch.

5.1.2 Baselines. To evaluate DECODE's effectiveness, we compared it with several state-of-the-art approaches selected based on the following criteria: ① The approaches must focus on operator-level testing. ② They should utilize constraint-driven testing, ensuring alignment with DECODE's methodology. Using these criteria, we identified three cutting-edge DL testing methods as baselines, each showcasing a distinct and valuable testing methodology for comparison.

- **DocTer** [48]: DocTer leverages natural language processing techniques to parse official framework documentation, extracting operator constraints and parameter specifications to guide test case generation.
- **IvySyn** [5]: IvySyn adopts a black-box fuzzing approach that uses type-aware fuzzing on low-level kernel code to discover memory error vulnerabilities and generates high-level code snippets to propagate error-triggering inputs.
- **DeepREL** [10]: DeepREL utilizes fuzzing to infer relational high-level APIs by examining APIs that share similar input and output patterns.

5.1.3 Experimental Configuration. All our experiments were conducted on a system running Ubuntu 22.04, equipped with an Intel(R) Xeon(R) Gold 5218R CPU @ 2.10 GHz and 128 GB of memory. For operator testing with DECODE, we allocated a maximum testing time of 10 minutes per operator, terminating execution early if an error was detected. Due to memory resource constraints, we limited DECODE's exploration range for `ndims` and `dim_size` to `[0, 10)`. In RQ2 and RQ3, baseline tools were evaluated using their default configurations. To ensure reliability in RQ2, we assessed pass rates and code coverage over five independent runs, averaging the results. Given the inherent randomness in test input generation, DECODE and IvySyn used pseudorandom seeds,

whereas DocTer and DeepREL relied on uncontrolled randomness. To account for this variability in RQ3, we conducted 10 independent runs per tool and aggregated the results to enable a fair and comprehensive comparison.

5.1.4 Bug Definition. We define a "bug" as any reproducible crash-inducing input that exposes memory safety violations or runtime failures in operator implementations. Specifically, a bug refers to an input that successfully passes front-end validation but triggers abnormal termination during backend operator execution, including: (1) Abort (unexpected termination), (2) FPE (floating-point exception), (3) SEGV (segmentation fault), and (4) Bus Error. We exclude ordinary input validation failures or expected error reports that do not lead to crashes.

5.2 RQ1 Extract Constraints

We employ DECODE to extract operator constraints from DL libraries and evaluate the quality of these constraints. We conduct a detailed analysis of the constraints generated, specifically focusing on Cross-Tensor Constraints and Tensor-Specific Constraints, which are generated in the dynamic exploration phase.

We randomly select 100 operators each from TensorFlow and PyTorch for manual analysis to evaluate the soundness and accuracy of the constraints extracted using DECODE. The manual analysis highlight several important considerations regarding the extracted constraints:

- **Cross-Tensor Constraints:** Some operator logics assume certain implicit relationships between input tensors that are not explicitly checked in the implementation. Listing 3 shows an example. FusedMatMul checks if inputs a and b are 2-dimension (see listing 4). If the assertion holds, it implicitly enforces the constraint that a and b have compatible dimensions, even though this constraint is not explicitly stated in the code. Our evaluation treats these implicit constraints as valid when analyzing the extracted constraints of DECODE.
- **Tensor-Specific Constraints:** To optimize performance, DECODE uses bounded exploration during the extraction of dimensions (*ndims*) and size ranges (*dim_size*). Specifically, the exploration is limited to the range [0, 10). These bounded results are treated as indicative of constraint satisfaction, representing a practical subset of the full range.

We assess the extracted constraints using precision, recall, and F1-score. Precision measures the percentage of correctly extracted constraints out of all extracted constraints. Recall represents the proportion of correctly extracted constraints relative to the total number of constraints present in the actual operator implementation. F1-score balances both metrics for a comprehensive evaluation. Note that not every operator has Cross-Tensor Constraints. Table 3 presents the evaluation of

Table 3. Manual Analysis Results and Analysis Cost Time for Constraint Extraction

	TensorFlow	PyTorch
Precision/Recall/F1 for All Constraints (%)	80.6/81.0/80.8	74.9/78.3/76.6
Precision/Recall/F1 for Cross-Tensor Constraints (%)	58.7/88.9/70.7	54.0/68.3/60.3
Precision/Recall/F1 for Tensor-Specific Constraints (%)	80.0/80.0/80.0	80.8/80.6/80.7
Analysis cost per operator (seconds)	17.5	24.3

precision, recall, and F1-score for the constraints extracted by DECODE, categorized into three groups: all constraints, Cross-Tensor constraints, and Tensor-Specific constraints for both TensorFlow and PyTorch. The last row of the table reports the average time required to extract operator constraints from TensorFlow and PyTorch, providing insights into the computational efficiency of the extraction process. DECODE demonstrates satisfactory performance in extracting constraints from both TensorFlow and PyTorch operators. For all constraints, DECODE achieves solid precision,

recall, and F1 scores, with results of 80.6%/81.0%/80.8% for TensorFlow and 74.9%/78.3%/76.6% for PyTorch. In terms of Cross-Tensor constraints, DECODE demonstrates impressive recall in both frameworks, achieving 88.9% for TensorFlow and 68.3% for PyTorch, though precision is slightly lower. For Tensor-Specific constraints, DECODE delivers a balanced performance with F1 scores around 80% in both frameworks, indicating effective constraint detection across different types. In terms of analysis time, DECODE requires an average of 17.5 seconds per operator for TensorFlow and 24.3 seconds per operator for PyTorch, highlighting the efficiency of the extraction process.

During the manual analysis of extracted constraints, we observe some issues, including incomplete extractions and false positives. The observed issues can be attributed to two main factors. Firstly, the dynamic exploration of operator constraints involves a degree of randomness in generating test inputs. As a result, there is no guarantee that the exploration process during the Dynamic Exploration phase will consistently align with expectations, potentially leading to incomplete or inaccurate constraint identification. Secondly, the formulation of Cross-Tensor Constraints is based on typical usage scenarios and lacks comprehensive consideration of broadcasting semantics², a mechanism that enables implicit alignment of tensor shapes with different ranks during element-wise operations. This oversight can result in incomplete constraint extraction, as the broadcasting behavior can introduce complexities that are not captured in the initial constraint definitions.

```
OP_REQUIRES(
    ctx, TensorShapeUtils::IsMatrix(a.shape()),
    errors::InvalidArgument("In[0] is not a matrix. Instead it has
        shape ", a.shape().DebugString()));
OP_REQUIRES(
    ctx, TensorShapeUtils::IsMatrix(b.shape()),
    errors::InvalidArgument("In[1] is not a matrix. Instead it has
        shape ", b.shape().DebugString()));
```

Listing 3. Assertions in tensorflow::FusedMatMul

```
class TensorShapeUtils {
    static bool IsMatrix(const TensorShape& shape) {
        return shape.dims() == 2; }
}
```

Listing 4. Implementation of function IsMatrix in TensorFlow. Function IsMatrix checks if a tensor's shape is two-dimensional.

Result for RQ1

DECODE demonstrates high efficiency in constraint extraction, taking only 17.5 seconds per operator for TensorFlow and 24.3 seconds for PyTorch. The extracted constraints achieve high quality, with precision/recall/F1 scores of 80.6%/81.0%/80.8% for TensorFlow and 74.9%/78.3%/76.6% for PyTorch.

5.3 RQ2 Test Case Quality

To demonstrate DECODE's effectiveness in generating inputs for DL operators, we evaluate its performance from two key metrics: the input passing rate and the code coverage. The input passing rate measures the proportion of generated inputs that successfully pass through operator-level assertion checks without triggering exceptions, reflecting DECODE's ability to produce valid inputs. Branch coverage assesses how well DECODE explores the various execution paths within operator implementations, highlighting its capacity to test diverse operator behaviors and edge cases.

5.3.1 Passing Rate. To effectively uncover bugs within operators, it is crucial that the generated inputs pass through the operator's assertion checks without causing exceptions. To evaluate DECODE's performance in generating valid inputs, we measure the percentage of inputs that passed through assertion checks and error-handling mechanisms during testing. This percentage reflects DECODE's ability to produce inputs that align with the expected behavior of the operator.

²<https://pytorch.org/docs/stable/notes/broadcasting.html>

For comparison, we benchmark DECODE's performance against three baseline tools: IvySyn, DocTer, and DeepREL.

Table 4. Comparison of Input Passing Ratios Across DocTer, IvySyn, DeepREL, and DECODE.

	DocTer	IvySyn	DeepREL	DECODE
TensorFlow	30.7%	1.3%	75.9%	40.1%
PyTorch	29.6%	4.7%	87.4%	36.2%

Table 5. Code Coverage Comparison (Line and Branch) Between DECODE and IvySyn.

	TensorFlow		PyTorch	
	Line cov	Branch cov	Line cov	Branch cov
DECODE	25,422	15,326	33,579	52,073
IvySyn	23,707	14,190	28,481	43,820
DocTer	14,105	8,447	10,558	16,402
DeepREL	16,684	11,686	20,224	24,708

Table 4 summarizes the input passing ratios for TensorFlow and PyTorch across four tools: DocTer, IvySyn, DeepREL, and DECODE. DECODE achieves the second highest input passing ratios, with 40.1% for TensorFlow and 36.2% for PyTorch, demonstrating its effectiveness in generating valid inputs. In comparison, DocTer achieves 30.7% and 29.6%, while IvySyn shows much lower passing ratios, with 1.3% and 4.7% for TensorFlow and PyTorch, respectively. DeepREL, utilizing historical input distributions, attains the highest passing ratios — 75.9% for TensorFlow and 87.4% for PyTorch. However, despite its high pass rates, DeepREL proves less effective in detecting vulnerabilities - uncovering only 9 bugs in total (see Section 5.4). This indicates that merely generating valid inputs is insufficient for robust bug detection. In contrast, DECODE strikes a better balance between input validity and test effectiveness, showcasing its strength in producing high-quality test cases capable of exposing deep-seated vulnerabilities in DL operators.

5.3.2 Code Coverage. To evaluate the effectiveness of exploring low-level operator logic, we measure *line* and *branch* coverage on the C++ implementations of TensorFlow and PyTorch operators. We compare DECODE against three baselines: IvySyn, which also targets C++ kernels directly, and DocTer and DeepREL, which generate inputs via the Python API but ultimately execute the same backend kernels. To ensure a fair comparison, our analysis explicitly excludes coverage metrics related to the inserted fuzzing harnesses, focusing solely on the native operator implementation files. This targeted scope eliminates interference from unrelated framework components (e.g., data loading or utility functions), providing a precise assessment of each tool's capability to exercise core operator functionalities.

Table 5 compares line and branch coverage results on TensorFlow and PyTorch. DECODE achieves higher coverage, with 25,422 lines and 15,326 branches covered in TensorFlow, and 33,579 lines and 52,073 branches covered in PyTorch. In comparison, IvySyn covers 23,707 lines and 14,190 branches in TensorFlow, and 28,481 lines and 43,820 branches in PyTorch. DECODE's coverage surpasses IvySyn by 1,715 lines and 1,136 branches in TensorFlow, and by 5,098 lines and 8,253 branches in PyTorch.

Although DocTer and DeepREL generate test inputs at the Python API level, we measure the operator-level coverage they ultimately induce. Their comparatively lower backend coverage suggests that documentation- or relation-guided input generation tends to concentrate on common execution paths and is less effective at exploring deep error-handling and boundary-condition logic in operator kernels. This observation aligns with their lower bug discovery rates reported in RQ3.

Result for RQ2

DECODE generates high-quality test cases using extracted constraints, achieving passing rates of 40.1% for TensorFlow and 36.2% for PyTorch - far surpassing IvySyn's 1.35% and 4.7%, respectively, due to its lack of constraint-guided test generation. Code coverage analysis shows that DECODE covers 25,422 lines and 15,326 branches in TensorFlow, and 33,579 lines and 52,073 branches in PyTorch, consistently outperforming IvySyn.

5.4 RQ3 Bug Finding Capability

In our study, a "bug" is defined as any input that reproducibly triggers memory errors (Abort, FPE, SEGV, Bus Error) in operator implementations. We report both total bugs (96) and developer-confirmed bugs (82, representing 47 unique defects) to provide comprehensive evaluation while enabling comparison with prior work.

5.4.1 Comparison with State-of-the-Art Testing Tools. To evaluate DECODE's bug finding capability in DL libraries, we conducted testing on TensorFlow and PyTorch. Table 6 summarizes the bug detection results of DECODE, IvySyn, DocTer, and DeepREL. DECODE outperforms all other tools, identifying 96 bugs, compared to 55 by IvySyn, 21 by DocTer, and only 9 by DeepREL. In TensorFlow, DECODE detected 54 bugs, doubling IvySyn's count (27 bugs) and exceeding DocTer's by over 2.5 times (21 bugs), while DeepREL found only 8 bugs. In PyTorch, DECODE identified 42 bugs, achieving a 50% increase compared to IvySyn (28 bugs), whereas DocTer found none and DeepREL only a single Bus Error. These results highlight DECODE's effectiveness in identifying diverse and critical bugs, showcasing a significant improvement compared to existing tools.

Table 6. Bug Detection Results Categorized by Error Types: Abort (Unexpected Termination), FPE (Floating-Point Exception), SEGV (Segmentation Fault), and Bus Error.

Tools	TensorFlow					PyTorch					Total
	Abort	FPE	SEGV	Bus Error	Total	Abort	FPE	SEGV	Bus Error	Total	
DECODE	42	2	10	0	54	6	8	28	0	42	96
IvySyn	22	1	4	0	27	5	5	18	0	28	55
DocTer	19	1	1	0	21	0	0	0	0	0	21
DeepREL	6	1	1	0	8	0	0	0	1	1	9

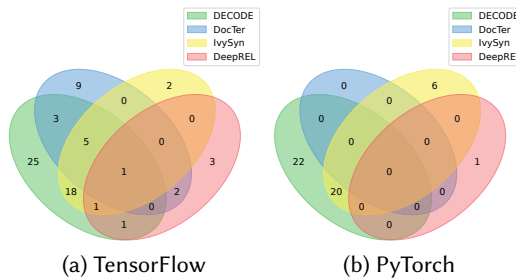


Fig. 6. Comparison of Bug Detection Numbers Across Tools for TensorFlow (a) and PyTorch (b)

Figure 6 compares the bugs detected by DECODE, IvySyn, DocTer, and DeepREL in TensorFlow and PyTorch, highlighting unique and overlapping findings. In the comparison with IvySyn,

Table 7. Error Distribution Between DECODE and IvySyn. Narrow Validation Scope refers to errors caused by assertions that either partially verify tensor properties or entirely omit necessary checks. Faulty Assertions are errors caused by flawed assertion logic, leading to crashes or exceptions during the execution of the assertion checks themselves. Improper Value Handling includes errors triggered by edge-case or unexpected values within tensor inputs, such as zero, 65,535, and 32,768. Derived Input Errors arise not from the initial inputs themselves, but from subsequent transformations or computations applied to them during operator execution. Other errors include failures caused by inputs that are not tensors, such as arrays, strings, or integers. (Note: "Difference" represents the discrepancy in the number of errors detected by DECODE compared to IvySyn. A positive value indicates that DECODE identified more errors of the same type than IvySyn.)

Error Type	TensorFlow			PyTorch		
	DECODE	IvySyn	Difference	DECODE	IvySyn	Difference
Narrow Validation Scope	19	12	+7	19	13	+7
Faulty Assertions	10	7	+3	0	0	0
Improper Value Handling	19	5	+14	1	0	+1
Derived Input Error	5	2	+3	3	1	+2
Other	1	1	0	19	14	+5
Total	54	27	+27	42	28	+15

DECODE detected 51 additional bugs that IvySyn missed (29 in TensorFlow and 22 in PyTorch) while sharing 45 bugs with IvySyn (25 in TensorFlow and 20 in PyTorch). Conversely, IvySyn found eight bugs that DECODE did not detect (two in TensorFlow and six in PyTorch). DECODE outperformed DocTer by detecting 87 additional bugs (45 in TensorFlow, 42 in PyTorch), sharing only nine bugs with DocTer in TensorFlow. DocTer detected 11 bugs not identified by DECODE, all in TensorFlow, and failed to find any bugs in PyTorch. Compared to DeepREL, DECODE found 51 and 42 more bugs in TensorFlow and PyTorch, respectively, while missing six bugs found by DeepREL. Additionally, DeepREL uncovered a bus error in PyTorch missed by all others.

5.4.2 Root Cause Analysis of Detected Vulnerabilities. To assess how the extracted constraints (see section 3.3) enhance DECODE's bug detection capabilities, we conducted a root cause analysis of detected vulnerabilities and compared them with those identified by IvySyn as detailed in Table 7. The table categorizes errors into five types: Narrow Validation Scope, Faulty Assertions, Improper Value Handling, Derived Input Error, and Other Errors. Among these, the first three are directly related to the constraints extracted by DECODE.

Since DECODE actively explores constraints to guide input generation, it is more effective at identifying vulnerabilities caused by faulty assertions or narrowly scoped validation logic. In contrast, IvySyn's black-box approach often causes assertion checks to block inputs before they can expose underlying vulnerabilities. As shown in Table 7, DECODE surpasses IvySyn in **Narrow Validation Scope**, where DECODE identifies 19 errors in TensorFlow versus IvySyn's 12, and 19 in PyTorch versus IvySyn's 13. Likewise, in **Faulty Assertions**, DECODE detects 10 errors in TensorFlow, outperforming IvySyn's 7, while neither tool identifies such errors in PyTorch.

Although DECODE does not explicitly model constraints on tensor values, its constraint-based approach ensures that inputs adhere to other key attributes, allowing for better exploration of tensor values and the detection of additional errors (see also passing rate comparison in section 5.3.1). This advantage is evident in **Improper Value Handling**, where DECODE uncovers 19 errors in TensorFlow compared to IvySyn's 4, and 1 error in PyTorch versus IvySyn's 0. Additionally, in **Derived Input Error**, DECODE detects 5 errors in TensorFlow versus IvySyn's 2, and 3 in PyTorch versus IvySyn's 1. These results highlight DECODE's ability to dynamically explore and refine constraints, enabling it to generate high-quality test cases that expose errors stemming from

Table 8. Distribution of Detected Vulnerabilities Across Ablated DECODE Variants. Both Cross-Tensor Constraints and Feedback-Driven Refinement Are Essential for Effective Bug Discovery.

Error Type	TensorFlow			PyTorch		
	DECODE	DECODE ^{-CT}	DECODE ^{-FB}	DECODE	DECODE ^{-CT}	DECODE ^{-FB}
Narrow Validation Scope	19	11	0	19	10	0
Faulty Assertions	10	1	0	0	0	0
Improper Value Handling	19	6	0	1	0	0
Derived Input Error	5	1	0	3	0	0
Other	1	1	0	19	14	7
Total	54	20	0	42	24	7

vulnerabilities caused by faulty assertions or narrowly scoped validation logic - issues that other tools like IvySyn often miss.

5.4.3 Ablation Study. To quantify the contribution of key design components in DECODE, we conduct an ablation study by selectively disabling individual mechanisms while keeping all other settings unchanged. Specifically, we evaluate the impact of: (i) **cross-tensor constraint modeling** and (ii) **feedback-driven constraint refinement**. All ablated variants are executed under the same experimental settings as the full DECODE, including identical operator sets, time budgets, and random seeds.

- DECODE^{-CT} (without Cross-Tensor Constraints). This variant removes all cross-tensor constraints during construction. Only intra-tensor constraints (e.g., rank bounds, per-dimension ranges, and dtype validity) are retained.
- DECODE^{-FB} (without Feedback-Driven Refinement). This variant disables feedback-driven constraint refinement. Extracted constraints remain fixed throughout fuzzing, and runtime assertion signals are not used for constraint updates.

Table 8 reports the distribution of detected errors under the full DECODE system and its two ablated variants. The results demonstrate that both cross-tensor constraint and feedback policy are essential for effective vulnerability discovery.

Impact of Cross-Tensor Constraints. Removing cross-tensor constraints (DECODE^{-CT}) causes substantial degradation in bug detection capability. For TensorFlow, the number of detected vulnerabilities decreases from 54 to 20 (63% reduction); for PyTorch, it drops from 42 to 24. The reduction is particularly evident in error categories that rely on cross-input reasoning, such as Narrow Validation Scope, Improper Value Handling, and Derived Input Error, indicating the importance of modeling relational constraints across inputs.

DECODE identified a bug in TensorFlow’s FakeQuantWithMinMaxVarsPerChannelGradient operator that DECODE^{-CT} failed to detect. As shown in Listing 6, the kernel implementation assumes that input has at least one dimension and unconditionally accesses its last dimension via `input.dim_size(input.dims()-1)`. When input is a scalar, this expression evaluates to an invalid index `-1`, resulting in undefined behavior and typically causing a crash. Although the operator enforces `input.IsSameSize(gradient)`, this check only ensures equality in the number of elements and does not prevent rank-0 tensors (see line 8 in listing 6).

DECODE^{-CT} fails to expose this vulnerability because it lacks cross-tensor constraint modeling. In the canonical test case (Listing 5), both gradients and inputs are initially inferred as rank 2; DECODE^{-CT} then varies each tensor’s rank independently, and all single-tensor variations violate the `IsSameSize` assertion and are rejected before reaching the kernel. By contrast, DECODE models

³Test case from the TensorFlow repository: https://github.com/tensorflow/tensorflow/blob/2aa7f9e5a154cec6ba4b37c04158042c99071348/tensorflow/python/kernel_tests/quantization_ops/quantization_ops_test.py#L110.

```
import tensorflow as tf

gradients = tf.constant(value=[[1.0], [2.0], [4.0]], shape=[3, 1],
                        dtype=dtypes.float32)
inputs = tf.constant(value=[[1.0], [2.0], [4.0]], shape=[3, 1], dtype=
                        dtypes.float32)
tf.raw_ops.FakeQuantWithMinMaxVarsPerChannelGradient(
    gradients=gradients, inputs=inputs, min=[[0.0]], max=[[1.0]])
```

Listing 5. TensorFlow official test case of FakeQuantWithMinMaxVarsPerChannelGradient³

```
class FakeQuantWithMinMaxVarsPerChannelGradientOp : public
    OpKernel {
    void Compute(OpKernelContext* context) override {
        CHECK_EQ(4, context->num_inputs());
        const Tensor& gradient = context->input(0);
        const Tensor& input = context->input(1);
        OP_REQUIRES(context, input.IsSameSize(gradient),
            InvalidArgument("gradient and input must be
                            the same size"));
        const int depth = input.dim_size(input.dims() - 1) ;
        ...
    }
}
```

Listing 6. Implementation of FakeQuantWithMinMaxVarsPerChannelGradient

the `IsSameSize` relation and can generate coordinated rank-0 inputs (both tensors scalar) that satisfy the assertion and trigger the downstream index-out-of-bounds crash. This coordinated-generation capability explains the 63% drop in detected vulnerabilities when cross-tensor constraints are removed (Table 8).

Impact of Feedback-Driven Refinement. Disabling the feedback-driven refinement ($\text{DECODE}^{-\text{FB}}$) almost completely eliminates DECODE's vulnerability discovery capability. Across all tensor-related error categories, the number of detected issues drops to zero or near zero for TensorFlow and PyTorch. This result highlights that tensor constraint hypothesis is insufficient in practice: without iteratively relaxing overly strict constraints or strengthening weak ones based on runtime feedback, the generated inputs remain confined to safe execution paths.

Takeaway. These results show that DECODE's effectiveness relies on the combination of cross-tensor reasoning and feedback-driven refinement. Cross-tensor constraints enable coordinated input construction, while feedback progressively corrects inaccurate initial constraints. Removing either component substantially reduces coverage and vulnerability discovery.

5.4.4 Failure Analysis. Although DECODE demonstrates strong bug finding capability, it fails to identify certain errors that are detected by baseline tools. Our analysis reveals three main reasons: 1) DECODE relies on constraint extraction from observed operator inputs. However, to maintain low overhead, its dynamic exploration phase generates a limited number of test inputs, which may leave portions of the constraint space unexplored. 2) DECODE builds its constraint models around general patterns observed in assertion checks. As a result, it may overlook operator-specific behaviors - such as broadcasting rules or dimension alignment - leading to missed bugs in those contexts. 3) The stochastic nature of DECODE's input generation can result in inconsistent coverage. It may fail to produce the precise input patterns required to expose subtle bugs or reach rare execution paths.

Result for RQ3

DECODE effectively uncovers 96 previously unknown bugs across TensorFlow and PyTorch, with 47 uniquely discovered by DECODE. Compared to baselines, DECODE demonstrates a significant advantage - detecting 41 more bugs than IvySyn, 75 more than DocTer, and 87 more than DeepREL. Its constraint-guided input generation notably enables detection of 24 more TensorFlow and 8 more PyTorch assertion-related bugs than IvySyn.

6 Discussion

6.1 Implications of Findings

DECODE utilizes dynamic exploration to extract the actual input constraints of operators, enabling more comprehensive testing. This method helps developers identify errors that may have been overlooked during the development process, such as the absence of input assertion checks, insufficiently strict checks, or other potential issues in the code. Furthermore, by examining the discovered errors, DECODE can highlight discrepancies or omissions in the documentation, ensuring that the actual code behavior aligns with the documented constraints. This process not only improves the reliability of DL libraries but also supports better documentation practices by identifying areas where the documentation might have failed to capture the full range of possible input conditions and operator behavior.

6.2 Generalizability

DECODE's methodology can be applied to various DL frameworks by targeting the core computational layer shared across DL architectures. These architectures consist of low-level hardware-optimized kernels, binding layers interfacing with programming languages, and high-level APIs for model development. DECODE focuses on kernel implementations, employing a dynamic exploration-based approach to accurately model constraints based on runtime behavior, rather than static analysis or official document extraction. This adaptability allows DECODE to capture subtle, implementation-specific behaviors and constraints, ensuring its effectiveness across diverse frameworks and backends.

7 Related Work

We have discussed related work on DL library testing in Section 2.1. To thoroughly test DL libraries, researchers have explored techniques such as fuzzing, Python program testing, and differential testing to uncover vulnerabilities and improve robustness. Given the unique input data structures in DL libraries, grammar-based fuzzing [14, 19, 38, 49] has emerged as a promising approach for generating valid structured inputs that satisfy operator syntax and constraints. G-EvoSuite [28] combines grammar-based fuzzing with search-based generation for effective JSON parser testing. Gandalf [25] applies generation-based fuzzing to DL libraries using context-free grammars [7] to produce syntactically correct inputs. In this paper, we employ grammar-based input generation to construct structured inputs (e.g., tensors, scalars, matrices) that satisfy the shape, dimension, and data type constraints of DL operators.

To further investigate DL libraries, several fuzzing-based testing approaches have been proposed. SkipFuzz [18] applies active learning to infer input constraints during fuzzing, while Orion [37] combines guided and corner-case input generation. EAGLE [43] employs differential testing to verify output consistency across APIs, data types, and optimizations. While these techniques have advanced DL library testing, they often lack a systematic focus on analyzing operator-specific input constraints, particularly Cross-Tensor Constraints, crucial for capturing dependencies between multiple tensors. In contrast, DECODE analyzes both Tensor-Specific and Cross-Tensor Constraints to enable more targeted exploration of operator behaviors and vulnerability detection.

8 Conclusion

We proposed DECODE, a dynamic exploration-driven testing framework, which can effectively model input constraints to guide DL operator testing. Specifically, DECODE intercepts valid inputs before the execution of DL operators to construct constraint hypotheses. It then further validates and refines constraints by systematically exploring input ranges and relationships.

We evaluated DECODE's performance on two mainstream DL libraries, PyTorch and TensorFlow, and compared it with three state-of-the-art approaches, IvySyn, DocTer, and DeepREL. In total, DECODE successfully identified 96 bugs, 82 of which were confirmed by the developers. Compared to state-of-the-art approaches, DECODE demonstrated a significant improvement in bug detection, detecting 41 more bugs than IvySyn, 75 more than DocTer, and 87 more than DeepREL. Furthermore, DECODE uncovered 25 unique bugs in TensorFlow and 22 unique bugs in PyTorch compared with baseline tools, demonstrating its efficiency in identifying hidden bugs.

9 Data Availability

This research follows Open Science principles, ensuring accessibility, transparency, and reproducibility. All discovered DL library bugs are publicly disclosed, and the DECODE code is released to support replication. The identified bugs are available at <https://github.com/DECODE-paper/DECODE>.

References

- [1] 2023. CVE-2023-25661. Common Vulnerabilities and Exposures (CVE). <https://www.cve.org/CVERecord?id=CVE-2023-25661>
- [2] 2023. CVE-2023-25668. Common Vulnerabilities and Exposures (CVE). <https://www.cve.org/CVERecord?id=CVE-2023-25668>
- [3] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: a system for Large-Scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.
- [4] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). doi:10.48550/arXiv.2303.08774
- [5] Neophytos Christou, Di Jin, Vaggelis Atlidakis, Baishakhi Ray, and Vasileios P Kemerlis. 2023. IvySyn: Automated Vulnerability Discovery in Deep Learning Frameworks. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2383–2400.
- [6] Roman Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*. 160–167. doi:10.1145/1390156.1390177
- [7] Armin Cremers and Seymour Ginsburg. 1975. Context-free grammar forms. *J. Comput. System Sci.* 11, 1 (1975), 86–117. doi:10.1016/S0022-0000(75)80051-1
- [8] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435. doi:10.1145/3597926.3598067
- [9] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3623343
- [10] Yinlin Deng, Chenyuan Yang, Anjiang Wei, and Lingming Zhang. 2022. Fuzzing deep-learning libraries via automated relational api inference. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 44–56. doi:10.1145/3540250.3549085
- [11] Zizhuang Deng, Guozhu Meng, Kai Chen, Tong Liu, Lu Xiang, and Chunyang Chen. 2023. Differential Testing of Cross Deep Learning Framework APIs: Revealing Inconsistencies and Vulnerabilities. In *32nd USENIX Security Symposium (USENIX Security 23)*. 7393–7410.
- [12] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining Incremental Steps of Fuzzing Research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association.
- [13] Gwihwan Go, Chijin Zhou, Quan Zhang, Xiazijian Zou, Heyuan Shi, and Yu Jiang. 2024. Towards More Complete Constraints for Deep Learning Library Testing via Complementary Set Guided Refinement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1338–1350. doi:10.1145/3650212.3680364
- [14] Patrice Godefroid, Hila Peleg, and Rishabh Singh. 2017. Learn&fuzz: Machine learning for input fuzzing. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 50–59. doi:10.1109/ASE.2017.8115618
- [15] Jiazhen Gu, Xuchuan Luo, Yangfan Zhou, and Xin Wang. 2022. Muffin: Testing deep learning libraries via neural architecture fuzzing. In *Proceedings of the 44th International Conference on Software Engineering*. 1418–1430. doi:10.

- 1145/3510003.3510092
- [16] Qianyu Guo, Xiaofei Xie, Yi Li, Xiaoyu Zhang, Yang Liu, Xiaohong Li, and Chao Shen. 2020. Audee: Automated testing for deep learning frameworks. In *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*. 486–498. doi:10.1145/3324884.3416571
 - [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778. doi:10.1109/CVPR.2016.90
 - [18] Hong Jin Kang, Pattarakrit Rattanukul, Stefanus Agus Haryono, Truong Giang Nguyen, Chaiyong Ragkhitwetsagul, Corina Pasareanu, and David Lo. 2022. Skipfuzz: Active learning-based input selection for fuzzing deep learning libraries. *arXiv preprint arXiv:2212.04038* (2022). doi:10.48550/arXiv.2212.04038
 - [19] Hyungseok Kim, Soomin Kim, Jungwoo Lee, and Sang Kil Cha. 2024. AsFuzzer: Differential Testing of Assemblers with Error-Driven Grammar Inference. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1099–1111. doi:10.1145/3650212.3680345
 - [20] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 75–86. doi:10.1109/CGO.2004.1281665
 - [21] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature* 521, 7553 (2015), 436–444. doi:10.1038/nature14539
 - [22] Guanpeng Li, Siva Kumar Sastry Hari, Michael Sullivan, Timothy Tsai, Karthik Pattabiraman, Joel Emer, and Stephen W Keckler. 2017. Understanding error propagation in deep learning neural network (DNN) accelerators and applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12. doi:10.1145/3126908.3126964
 - [23] Meiziniu Li, Jialun Cao, Yongqiang Tian, Tsz On Li, Ming Wen, and Shing-Chi Cheung. 2023. Comet: Coverage-guided model generation for deep learning library testing. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–34. doi:10.1145/3583566
 - [24] Yi Li, Shaohua Wang, and Tien N Nguyen. 2022. UTANGO: untangling commits with context-aware, graph-based, code change clustering learning model. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 221–232. doi:10.1145/3540250.3549171
 - [25] Jiawei Liu, Yuheng Huang, Zhijie Wang, Lei Ma, Chunrong Fang, Mingzheng Gu, Xufan Zhang, and Zhenyu Chen. 2023. Generation-based Differential Fuzzing for Deep Learning Libraries. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–28. doi:10.1145/3628159
 - [26] Weisi Luo, Dong Chai, Xiaoyue Ruan, Jiang Wang, Chunrong Fang, and Zhenyu Chen. 2021. Graph-based fuzz testing for deep learning inference engines. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 288–299. doi:10.1109/ICSE43902.2021.00037
 - [27] Chenyang Lyu, Jiacheng Xu, Shouling Ji, Xuhong Zhang, Qinying Wang, Binbin Zhao, Gaoning Pan, Wei Cao, Peng Chen, and Raheem Beyah. 2023. MINER: A Hybrid Data-Driven Approach for REST API Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4517–4534.
 - [28] Mitchell Olsthorn, Arie van Deursen, and Annibale Panichella. 2020. Generating highly-structured input data by combining search-based testing and grammar-based fuzzing. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 1224–1228. doi:10.1145/3324884.3418930
 - [29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
 - [30] Konstantinos Pelechrinis, Marios Iliofotou, and Srikanth V Krishnamurthy. 2010. Denial of service attacks in wireless networks: The case of jammers. *IEEE Communications surveys & tutorials* 13, 2 (2010), 245–257. doi:10.1109/SURV.2011.041110.00022
 - [31] Hung Viet Pham, Thibaud Lutellier, Weizhen Qi, and Lin Tan. 2019. CRADLE: cross-backend validation to detect and localize bugs in deep learning libraries. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1027–1038. doi:10.1109/ICSE.2019.00107
 - [32] Van-Thuan Pham, Marcel Böhme, Andrew E Santosa, Alexandru Răzvan Căciulescu, and Abhik Roychoudhury. 2019. Smart greybox fuzzing. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1980–1997. doi:10.1109/TSE.2019.2941681
 - [33] PyTorch. 2024. *PyTorch Tests*. PyTorch. <https://github.com/pytorch/pytorch/tree/v2.5.0/test>
 - [34] Dongdong She, Kexin Pei, Dave Epstein, Junfeng Yang, Baishakhi Ray, and Suman Jana. 2019. Neuzz: Efficient fuzzing with neural program smoothing. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 803–817. doi:10.1109/SP.2019.00052
 - [35] Jingyi Shi, Yang Xiao, Yuekang Li, Yeting Li, Dongsong Yu, Chendong Yu, Hui Su, Yufeng Chen, and Wei Huo. 2023. Acetest: Automated constraint extraction for testing deep learning operators. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 690–702. doi:10.1145/3597926.3598088

- [36] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024). doi:10.52202/075280-0377
- [37] Nima Shiri Harzevili, Mohammad Mahdi Mohajer, Moshi Wei, Hung Viet Pham, and Song Wang. 2024. History-Driven Fuzzing for Deep Learning Libraries. *ACM Trans. Softw. Eng. Methodol.* 34, 1, Article 19 (Dec. 2024), 29 pages. doi:10.1145/3688838
- [38] Prashast Srivastava and Mathias Payer. 2021. Gramatron: Effective grammar-aware fuzzing. In *Proceedings of the 30th acm sigsoft international symposium on software testing and analysis*. 244–256. doi:10.1145/3460319.3464814
- [39] Robert Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM journal on computing* 1, 2 (1972), 146–160. doi:10.1137/0201010
- [40] Robert E Tarjan and Jan Van Leeuwen. 1984. Worst-case analysis of set union algorithms. *Journal of the ACM (JACM)* 31, 2 (1984), 245–281. doi:10.1145/62.2160
- [41] TensorFlow. 2024. *TensorFlow Tests*. TensorFlow. https://github.com/tensorflow/tensorflow/tree/v2.16.1/tensorflow/python/kernel_tests
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS’17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [43] Jiannan Wang, Thibaud Lutellier, Shangshu Qian, Hung Viet Pham, and Lin Tan. 2022. EAGLE: creating equivalent graphs to test deep learning libraries. In *Proceedings of the 44th International Conference on Software Engineering*. 798–810. doi:10.1145/3510003.3510165
- [44] Xu Wang, Sen Wang, Xingxing Liang, Dawei Zhao, Jincai Huang, Xin Xu, Bin Dai, and Qiguang Miao. 2022. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (2022), 5064–5078. doi:10.1109/TNNLS.2022.3207346
- [45] Zan Wang, Ming Yan, Junjie Chen, Shuang Liu, and Dongdi Zhang. 2020. Deep learning library testing via effective model generation. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 788–799. doi:10.1145/3368089.3409761
- [46] Anjiang Wei, Yinlin Deng, Chenyuan Yang, and Lingming Zhang. 2022. Free lunch for testing: Fuzzing deep-learning libraries from open source. In *Proceedings of the 44th International Conference on Software Engineering*. 995–1007. doi:10.1145/3510003.3510041
- [47] Qixue Xiao, Kang Li, Deyue Zhang, and Weilin Xu. 2018. Security risks in deep learning implementations. In *2018 IEEE Security and privacy workshops (SPW)*. IEEE, 123–128. doi:10.1109/SPW.2018.00027
- [48] Danning Xie, Yitong Li, Mijung Kim, Hung Viet Pham, Lin Tan, Xiangyu Zhang, and Michael W Godfrey. 2022. Docter: Documentation-guided fuzzing for testing deep learning api functions. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 176–188. doi:10.1145/3533767.3534220
- [49] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*. 283–294. doi:10.1145/1993498.1993532
- [50] Yuhao Zhang, Yifan Chen, Shing-Chi Cheung, Yingfei Xiong, and Lu Zhang. 2018. An empirical study on tensorflow program bugs. In *Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis*. 129–140. doi:10.1145/3213846.3213866
- [51] Chijin Zhou, Quan Zhang, Mingzhe Wang, Lihua Guo, Jie Liang, Zhe Liu, Mathias Payer, and Yu Jiang. 2022. Minerva: browser API fuzzing with dynamic mod-ref analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1135–1147. doi:10.1145/3540250.3549107

Received 2025-09-11; accepted 2026-03-24